Process Flow Document

Retail Demand Forecasting — Walmart Store Sales (Time-Series Models)

A technical blueprint of the data science pipeline — covering ingestion, cleaning, feature engineering, model training/validation, and prediction/post-processing — reproducible end-to-end by another data scientist.

Deliverable 3.2 — Process Flow Document (15%)
Source notebook: walmart_demand_forecasting_timeseries.ipynb

Objective

Retailers such as Walmart require accurate weekly sales forecasts to plan inventory, staffing, and promotions. Sales are influenced by seasonality, holidays, markdown promotions, and macro-economic conditions. Because ARIMA, SARIMA, and Prophet are univariate models, Store/Dept-level records are aggregated into a single company-wide weekly demand series; running a separate ARIMA model for every Store × Dept combination (45 × 81 = 3,645 series) is out of scope. LSTM and GRU forecast the same aggregate series but additionally take macro and holiday features as input channels.

This document is a stage-by-stage technical blueprint of the pipeline implemented in the companion notebook. It is written so that another data scientist could reproduce the work without access to the original code.

Key Corrections Applied in This Version

- **No target-derived leakage:** no aggregate such as a per-store average sales figure is computed before the train/test split; every transform (scaling, decomposition, holiday list) is fit on the training portion only.
- **Negative sales handled explicitly:** 1,285 rows (0.305% of data) had negative Weekly_Sales; these are investigated and clipped at 0 rather than silently passed through.
- **Missing values resolved before aggregation:** MarkDown1–5 NaNs are filled with 0 before any weekly aggregation, and a NaN check gates every model input.
- **Strictly chronological split:** the final 13 weeks are held out as the test set; no random shuffling is used at any stage.
- **Full statistical verification:** ADF, KPSS, seasonal decomposition, ACF/PACF, and Ljung-Box residual diagnostics are run before and after each classical model.

End-to-End Process Flow Diagram

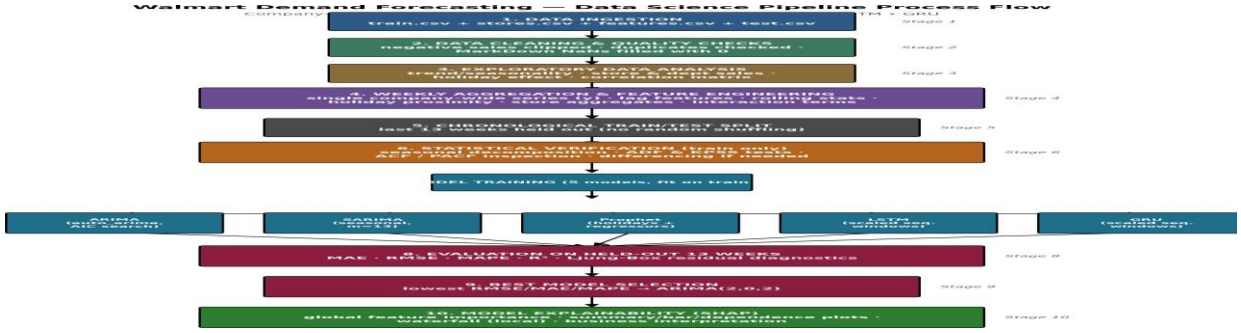

Figure 1. Ten-stage pipeline from raw CSV ingestion through model explainability. Data flows top-to-bottom; Stage 7 fans out into five independently trained forecasting models that reconverge at evaluation.

1. Data Ingestion

Four source files are loaded independently and then merged into modeling frames. **train.csv** and **test.csv** hold Store/Dept/Date-level Weekly_Sales history and the forecast horizon respectively; **features.csv** supplies date-aligned exogenous signals (Temperature, Fuel_Price, MarkDown1–5, CPI, Unemployment, IsHoliday); **stores.csv** supplies static store attributes (Type, Size).

1.1 Loading

```
train = pd.read_csv(r"/content/train.csv")
stores = pd.read_csv(r"/content/stores.csv")
features = pd.read_csv(r"/content/features.csv")
test = pd.read_csv(r"/content/test.csv")
```

1.2 Merging

train_df is built by left-joining train with features on the composite key [Store, Date, IsHoliday], then left-joining the result with stores on Store. The same two joins are applied to test to produce test_df. Left joins are used throughout so that no sales record is dropped if an exogenous value happens to be missing for that date/store
— the resulting frame is 421,570 rows × 16 columns for the training portion.

1.3 Validation Checks Performed on Ingest

- Row/column count and dtype audit via df.info().
- Per-column missing-value count via df.isnull().sum() — confirms MarkDown1–5 are the only columns with material missingness (64–74% missing, since markdown promotions did not run in all store-weeks).
- Duplicate-row check via df.duplicated().sum()— result: 0 duplicates.
- Date field is cast to datetime64 immediately after merge so every downstream operation can rely on chronological ordering.

2. Data Cleaning & Transformation

2.1 Negative Weekly_Sales (Outlier Handling)

1,285 rows (0.305% of the data) contain negative Weekly_Sales, ranging from −0.02 to −4,988.94 (mean −68.61), most plausibly representing returns exceeding gross sales in that store-week. These are investigated explicitly rather than dropped, and then clipped at a floor of 0 with df['Weekly_Sales'].clip(lower=0), since demand cannot be meaningfully negative once aggregated to a weekly total for a forecasting target. A post-clip assertion confirms zero negative values remain.

2.2 Missing MarkDown Values

MarkDown1–5 are promotional markdown amounts that are NaN whenever no markdown event occurred; NaN is therefore treated as “no markdown” and filled with 0 (fillna(0)). This is done immediately after merge and before any aggregation or EDA step touches the columns, avoiding a bug in the prior version of this notebook where NaNs survived several EDA steps before being resolved deep inside feature engineering. A derived column, Total_MarkDown, sums MarkDown1–5 row-wise. A NaN-count assertion gates this step.

2.3 Categorical Encoding

IsHoliday (boolean) is cast to integer (0/1) once the series is aggregated to weekly grain, so it can be used directly as a regressor/feature in every downstream model. Store Type (A/B/C) is retained as a categorical field for EDA (average sales by type) but is not carried into the company-wide aggregate series, since the forecasting target is a single national total rather than a per-store or per-type series.

2.4 Duplicate & Completeness Checks

After merging, cleaning, and aggregating to a weekly series, the pipeline asserts that the resulting weekly date index is complete: the expected number of weeks between the first and last date (a full pd.date_range at weekly frequency) is compared against the actual row count of the aggregated frame (143 expected vs. 143 actual, with 0 residual NaNs across all columns) before any model sees the data.

3. Feature Engineering Pipeline

All Store/Dept-level rows are first aggregated to a single weekly series (grouped by Date), with Weekly_Sales summed to the target **y** and exogenous fields aggregated by mean (Temperature, Fuel_Price, CPI, Unemployment) or sum/max (Total_MarkDown, IsHoliday). Every feature engineered afterward is derived from this weekly frame, and lag/rolling features are shifted so that no feature for week t uses information observed at or after week t.

Feature	Definition	Rationale
y_lag_1, y_lag_2, y_lag_4, y_lag_52	Weekly_Sales shifted by 1, 2, 4, and 52 weeks	Captures short-term momentum (t-1, t-2), monthly-scale (t-4), and annual seasonal recurrence (t-52) in demand.
y_roll_mean_4, y_roll_std_4	Rolling mean/std of y over the trailing 4 weeks (shifted by 1 to avoid leakage)	Smooths short-term noise and quantifies recent volatility, both predictive of near-term demand level and uncertainty.
y_roll_mean_12, y_roll_std_12	Rolling mean/std of y over the trailing 12 weeks (shifted by 1)	Captures a quarter-length trend/volatility baseline, complementing the shorter 4-week window.
weeks_to_holiday	Minimum distance in weeks from the current date to the nearest known holiday date	Encodes holiday proximity as a continuous ramp rather than a binary flag, since sales build up and taper around holidays rather than jumping discretely on the holiday week alone.
store_mean_sales, store_std_sales	Per-store mean and standard deviation of Weekly_Sales, merged back onto the row-level frame	Store-level aggregation that characterizes the scale and variability of individual stores, useful context even though the final series is company-wide.
temp_x_fuel	Temperature × Fuel_Price	Interaction term capturing combined macro/seasonal purchasing conditions not represented by either variable alone.
markdown_x_holid ay	Total_MarkDown × IsHoliday	Interaction term isolating the incremental effect of markdown promotions that specifically coincide with holiday weeks.
Total_MarkDown	Row-wise sum of MarkDown1–MarkDown5 (post-fill)	Single consolidated promotional-intensity signal, simpler for models and SHAP interpretation than five separate sparse columns.

After engineering, missing values introduced by lag/rolling shifts are limited to the first few rows of the series (e.g., y_lag_52 is null for <0.01% of rows, corresponding to weeks before a full year of history existed) and are handled by the model-specific windowing logic described in Section 4 rather than by imputation, since these are structurally unavailable at the start of the series rather than data-quality gaps.

4. Model Training & Validation

4.1 Train/Test Split Strategy

The weekly series (143 weeks total) is split strictly chronologically: the last 13 weeks (2012-08-03 through 2012-10-26) are held out as the test set, and the preceding 130 weeks (2010-02-05 through 2012-07-27) form the training set. A random train/test split is deliberately avoided because it would let the model train on weeks that occur after test weeks in time, silently leaking future trend and seasonal information into training and producing an optimistic, non-representative error estimate. Every scaler, holiday list, and seasonal decomposition used later in the pipeline is fit on the training portion only and then applied to the test portion.

4.2 Pre-Modeling Statistical Verification (train set only)

- **Seasonal decomposition** (additive, period=13) separates trend, seasonal, and residual components and visually confirms a recurring annual spike around the November/December holiday period.
- **Augmented Dickey-Fuller (ADF) test** on the level series: statistic = −5.64, p = 0.0000 → rejects the unit-root null, series is stationary.
- **KPSS test** on the level series: statistic = 0.068, p = 0.10 → fails to reject the stationarity null, consistent with the ADF result.
- Both tests agree the level series is already stationary, so d = 0 is appropriate; a first-difference is nonetheless computed and re-tested as a sanity check (ADF p = 0.0000, KPSS p = 0.10, both stationary) before finalizing model order.
- **ACF/PACF plots** on the differenced series provide a manual cross-check against the (p, d, q) orders that auto_arima selects.

4.3 Models Implemented

Five forecasting models are trained on the identical 130-week training window and evaluated on the identical 13-week held-out window:

Model	Configuration	Hyperparameter Search
ARIMA	Non-seasonal, with and without exogenous regressors (Temperature, Fuel_Price, CPI, Unemployment, Total_MarkDown)	pmdarima.auto_arima, stepwise search minimizing AIC; selected order (2,0,2)
SARIMA	Seasonal ARIMA, seasonal period m=13 (a practical quarterly-scale period chosen because only ~130 training weeks are available — too few for a full m=52 annual search)	auto_arima with seasonal=True, bounded search (max_p=3, max_q=3, max_P=2, max_Q=2); selected order (2,0,2)(2,0,1,13)
Prophet	Additive model with a custom holiday calendar built from IsHoliday dates (train period only) and the same five exogenous columns added as external regressors; yearly seasonality on, weekly/daily off	Regressor set and holiday window fixed by domain knowledge; no automated search performed for this model
LSTM	2-layer stacked LSTM (64 → 32 units, tanh, dropout 0.2 each) + Dense(1) output, one-step-ahead sequence-to-value	EarlyStopping (patience=8, restore_best_weights=True) over up to 100 epochs; batch size 8; converged after 29 epochs
GRU	2-layer stacked GRU (64 → 32 units, tanh, dropout 0.2 each) + Dense(1) output, identical architecture depth to the LSTM for a like-for-like comparison	Same EarlyStopping/epoch/batch settings as LSTM; converged after 8 epochs

4.4 Sequence Construction for LSTM/GRU (Leakage-Safe Windowing)

The target and six exogenous columns are scaled with a MinMaxScaler that is fit on the training rows only (never on test or the full series) and then applied to the entire series. Sliding windows of length LOOKBACK = 8 weeks of actual observed values predict the next single week (a one-step-ahead setup), which is the standard leakage-safe way to evaluate a sequence model on a short series. Training windows are built entirely from indices inside the training period (122 sequences of shape 8 × 7); test windows are built so their final LOOKBACK actuals may extend into the tail of the training period only as needed to produce a target that falls in the test period (13 sequences).

4.5 Validation Approach

Because the deliverable holds out a single continuous 13-week block at the end of the series, model comparison here uses one chronological holdout rather than a multi-fold rolling/expanding-window cross-validation; the same principle (train strictly precedes test in time, refit only on data available up to each cutoff) is what a rolling/expanding-window CV would enforce across multiple folds, and is noted as a recommended extension in Section 3.4 of the evaluation deliverable for more robust error estimates.

5. Prediction & Post-Processing

5.1 Generating Predictions

- **ARIMA / SARIMA:** model.predict(n_periods=13) produces point forecasts (and confidence intervals for ARIMA) directly on the original Weekly_Sales scale — no inverse-transform is needed since these models are fit on unscaled data.
- **Prophet:** a future frame carrying the test period's actual Temperature/Fuel_Price/CPI/Unemployment/Total_MarkDown values (legitimately available, since Walmart's features.csv already supplies known macro data for the forecast horizon, not a target leak) is passed to model.predict(), returning yhat plus uncertainty intervals and trend/seasonal/holiday components.
- **LSTM / GRU:** predictions are produced in scaled [0,1] space and must be inverse-transformed back to raw sales dollars using the same MinMaxScaler fit in Section 4.4, via an inverse_target helper that places the scaled prediction back into the correct feature column before calling scaler.inverse_transform.

5.2 Post-Processing

No ensembling of the five models is applied in this notebook — each model's forecast is evaluated independently so that its standalone performance is visible for the model-selection deliverable. The only post-processing steps applied to raw model output are: (a) inverse-scaling for the two neural models, and (b) alignment of every model's prediction array to the same 13-date test index before scoring, so MAE/RMSE/MAPE/R^2 are computed on directly comparable arrays across models.

5.3 Evaluation Metrics & Result

A single evaluate() helper computes MAE, RMSE, MAPE, and R^2 for each model against the held-out actuals and appends the result to a shared results table:

Model	MAE	RMSE	MAPE (%)	R^2
ARIMA	1,388,939.53	1,767,506.77	3.07	-0.44
SARIMA	1,464,710.11	1,928,347.35	3.24	-0.72
LSTM	1,787,817.33	2,163,275.10	3.94	-1.16
Prophet	1,913,532.62	2,318,128.28	4.14	-1.48
GRU	2,020,551.30	2,321,596.32	4.45	-1.49

ARIMA achieves the lowest error on every metric and is selected as the best-performing model on this holdout (full trade-off analysis, business context, and limitations are addressed in the companion Best Model Selection & Reasoning and Model Evaluation deliverables). Its residuals are additionally checked with a Ljung-Box test ($p = 0.0016$ at lag 10), which flags some remaining autocorrelation — consistent with the two large holiday-week spikes visible in the residual plot — while SARIMA's residuals pass the same test ($p = 0.187$), a trade-off discussed further in the evaluation deliverable.

5.4 Explainability as Post-Processing

For the selected ARIMA(X) model, SHAP (KernelExplainer-style, via shap.Explainer wrapping the model's exogenous-driven predict function) is applied to the exogenous regressors to produce: a summary/beeswarm plot, a bar chart of mean absolute SHAP value per feature, a waterfall plot decomposing one individual test-week prediction, a dependence plot for Temperature, a decision plot across the first five test weeks, and a ranked feature-importance table — translating the statistical model into stakeholder-facing explanations of which exogenous drivers move the forecast.

This document, together with the source notebook (walmart_demand_forecasting_timeseries.ipynb) and its requirements.txt, is sufficient for another data scientist to reproduce every stage of the pipeline described above.